

Worksheet No. - 04

Student Name: Sukhleen Kaur

Branch: MCA

Semester: 3rd

Subject Name: Automation Software Testing

UID: 25MCA20031

Section/Group: 25MCA-6B

Date of Performance: 16.09.2026

Subject Code: 25CAH-705

Aim/Overview of the practical: (A). Install and configure the development environment for web automation using Selenium and analyze the setup requirements in terms of dependencies, platform compatibility, and ease of configuration.

Objective: To install and configure the required development environment for web automation using Selenium, including Python, virtual environment, and Selenium libraries, while understanding the dependencies, platform compatibility, and configuration requirements needed for successful automation testing.

Procedure

1. Set Up Python Virtual Environment

- Open VS Code and open your project folder.

Open the Terminal from Terminal → New Terminal.

- Check that Python is installed:

```
python --version
```

- Create a virtual environment:

```
python -m venv venv
```

- Activate the virtual environment on Windows:

```
venv\Scripts\activate
```

- Check that (venv) appears at the beginning of the terminal.

2. Install Selenium Libraries

- Make sure the virtual environment is activated.
- Upgrade pip:

```
python -m pip install --upgrade pip
```

- Install Selenium:

pip install selenium

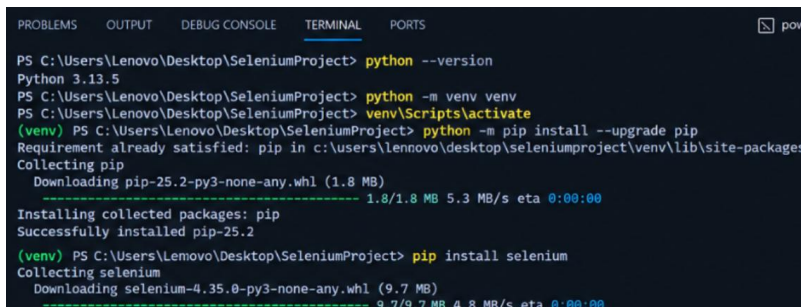
- Verify that Selenium is installed:

pip show selenium

- Selenium is now ready to be used for browser automation and testing.

Analysis:

Step	Activity	Analysis
1	Check Python	Python installation was verified.
2	Create Virtual Environment	venv was created successfully.
3	Activate Environment	The virtual environment was activated.
4	Install Selenium	Selenium was installed successfully.
5	Verify Installation	Selenium was verified and ready for testing.



```

PS C:\Users\Lenovo\Desktop\SeleniumProject> python --version
Python 3.13.5
PS C:\Users\Lenovo\Desktop\SeleniumProject> python -m venv venv
PS C:\Users\Lenovo\Desktop\SeleniumProject> venv\Scripts\activate
(venv) PS C:\Users\Lenovo\Desktop\SeleniumProject> python -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\lenovo\desktop\seleniumproject\venv\lib\site-packages
Collecting pip
  Downloading pip-25.2-py3-none-any.whl (1.8 MB)
----- 1.8/1.8 MB 5.3 MB/s eta 0:00:00
Installing collected packages: pip
Successfully installed pip-25.2
(venv) PS C:\Users\Lenovo\Desktop\SeleniumProject> pip install selenium
Collecting selenium
  Downloading selenium-4.35.0-py3-none-any.whl (9.7 MB)
----- 9.7/9.7 MB 4.8 MB/s eta 0:00:00

```

Result: The Python virtual environment was successfully created and activated. Selenium was installed and verified successfully, and the environment is ready for performing web browser automation and testing using Python.

Aim/Overview of the practical: (B). Develop and execute a selenium script to evaluate the search functionality of a web application (Amazon / Flipkart / Myntra).

Objective: To develop and execute a Selenium automation script to test and evaluate the search functionality of a web application such as Amazon, Flipkart, or Myntra by entering product keywords, performing searches, and verifying that relevant search results are displayed correctly.

Procedure/Code:

```
import time
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException
```

```
def test_souled_store_search(product: str):
    options = webdriver.ChromeOptions()
    options.add_argument("--start-maximized")
    options.add_argument("--disable-notifications")

    driver = webdriver.Chrome(options=options)
    wait = WebDriverWait(driver, 20)

    try:
        print("[INFO] Opening The Souled Store...")
        driver.get("https://www.thesouledstore.com/explore/backpacks")

        wait.until(EC.presence_of_element_located((By.TAG_NAME, "body")))
        time.sleep(3)

        print("[INFO] Website loaded successfully.")
        print("[INFO] Looking for search bar...")

        search_box = None
        inputs = driver.find_elements(By.TAG_NAME, "input")

        for element in inputs:
            try:
                if element.is_displayed() and element.is_enabled():
                    placeholder = element.get_attribute("placeholder")
                    aria_label = element.get_attribute("aria-label")
                    name = element.get_attribute("name")
                    element_type = element.get_attribute("type")

                    text_to_check = " ".join(
                        [
                            placeholder or "",
```

```
        aria_label or "",
        name or ""
    ]
).lower()
```

```
    if "search" in text_to_check or element_type == "search":
        search_box = element
        break
except Exception:
    continue
```

```
if search_box is None:
    print("[INFO] Search input is not visible.")
    print("[INFO] Looking for search button...")
```

```
search_buttons = driver.find_elements(
    By.XPATH,
    "//*[contains(@aria-label,'Search')]"
    " | //button[contains(@title,'Search')]"
    " | //a[contains(@aria-label,'Search')]"
    " | //a[contains(@title,'Search')]"
)
```

```
for button in search_buttons:
    try:
        if button.is_displayed() and button.is_enabled():
            driver.execute_script("arguments[0].click();", button)
            print("[INFO] Search button clicked.")
            break
    except Exception:
        continue
```

```
time.sleep(2)
inputs = driver.find_elements(By.TAG_NAME, "input")
search_box = None
```

```
for element in inputs:
    try:
        if element.is_displayed() and element.is_enabled():
            placeholder = element.get_attribute("placeholder")
            aria_label = element.get_attribute("aria-label")
            name = element.get_attribute("name")
            element_type = element.get_attribute("type")
```

```
text_to_check = " ".join(
    [
        placeholder or "",
        aria_label or "",
        name or "",
    ]
).lower()

if "search" in text_to_check or element_type == "search":
    search_box = element
    break
except Exception:
    continue

if search_box is None:
    raise Exception("Search bar could not be found.")

print("[INFO] Search bar found.")
driver.execute_script("arguments[0].scrollIntoView({block:'center'});", search_box)
time.sleep(1)
driver.execute_script("arguments[0].click();", search_box)

print(f"[INFO] Entering search keyword: {product}")
search_box.clear()
search_box.send_keys(product)
print(f"[INFO] '{product}' entered in search bar.")

time.sleep(2)
print("[INFO] Searching...")
search_box.send_keys(Keys.ENTER)
time.sleep(5)

print("\n[INFO] URL after search:")
print(driver.current_url)

product_links = driver.find_elements(
    By.XPATH, "//a[contains(@href, 'product/')]")
)

results = []
for link in product_links:
    try:
        product_name = link.text.strip()
        if product_name and product_name not in results:
```

```
results.append(product_name)
except Exception:
    continue
```

```
print("\nTHE SOULED STORE SEARCH TEST")
print(f'Search Keyword : {product}')
print(f'Products Found : {len(results)}')
```

```
if len(results) > 0:
    print("\nSearch Results:")
    for i, result in enumerate(results[:10], start=1):
        result = result.replace("\n", " - ")
        print(f'{i}. {result}')
    print("\n[SUCCESS] Search test PASSED.")
else:
    print("\n[FAIL] No products found for Accessories.")
```

```
except TimeoutException:
    print("[ERROR] Timeout while loading the website or search bar.")
except Exception as e:
    print(f'[ERROR] Unexpected error: {e}')
finally:
    print("\n[INFO] Closing browser...")
    time.sleep(3)
    driver.quit()
```

```
if __name__ == "__main__":
    product = "Accessories"
    test_souled_store_search(product)
```

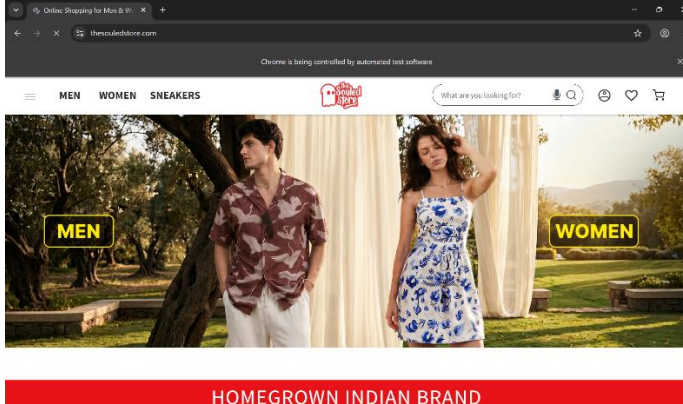
Test Cases:

TC ID	Test Case	Test Data	Expected Result	Actual Result	Status
TC_01	Verify The Souled Store website loads successfully	Website URL	Website should load successfully	Website loaded successfully	PASS
TC_02	Verify search bar is available	Search bar	Search bar should be visible and enabled	Search bar was found and enabled	PASS
TC_03	Verify user can enter a search keyword	Accessories	Search keyword should be entered successfully	Accessories was entered successfully	PASS
TC_04	Verify search functionality	Accessories	Relevant search results should be displayed	Products were found for Accessories	PASS

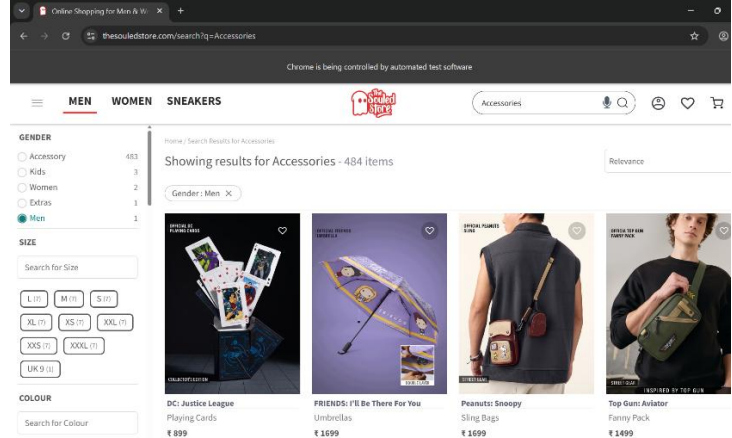
TC_05	Verify product results are displayed	Accessories	Product names should be displayed in the search results	Product names were retrieved and displayed	PASS
-------	--------------------------------------	-------------	---	--	------

Output

Home Page



Search Button



Terminal

```
[INFO] URL after search:
https://www.thesouledstore.com/search?q=Accessories
THE SOULED STORE SEARCH TEST
Search Keyword : Accessories
Products Found : 24

Search Results:
1. DC: Justice League - Playing Cards - ₹ 899
2. FRIENDS: I'll Be There For You - Umbrellas - ₹ 1699
3. Peanuts: Snoopy - Sling Bags - ₹ 1699
4. Top Gun: Aviator - Fanny Pack - ₹ 1499
5. Spider-Man: Web-Slinger - Caps - ₹ 999
6. Souled Socks: Blue Fire - Socks - ₹ 299
7. FRIENDS: Purple Frame - Toiletry Kit - ₹ 1899
8. Spider-Man: Webline - Bag Charms - ₹ 499
9. Powerpuff Girls: Blossom - Socks - ₹ 299
10. Avengers: The Kang Dynasty [New Printing] - Collectors Comic Books - ₹ 2999

[SUCCESS] Search test PASSED.
```

Learning Outcomes:

1. Understood the basics of Selenium WebDriver for web application testing.
2. Automated the search functionality of a web application.
3. Located and interact with web elements using XPath and Selenium locators.
4. Entered search keywords and verify the search results automatically.

Evaluation Grid:

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Demonstration and Performance		12
2.	Worksheet		8
3.	Viva		10